

Rule-based 控制器原理说明

面向初学者的项目与算法理解文档

MADRL_ESS 项目理解笔记

2026 年 7 月 27 日

目录

1	先建立直觉：这个项目到底在控制什么	3
2	基本符号和单位	3
3	EV 是否在家的时间规则	4
4	Rule-based 无电池版本	5
4.1	核心思想	5
4.2	为什么这个规则简单但不聪明	5
5	Rule-based 有电池版本	5
5.1	核心思想	5
5.2	进口电价	6
5.3	价格分位数阈值	6
5.4	电池功率规则	7
5.5	备用规则：中位数两段式	7
6	从功率请求到环境动作	7
6.1	电池动作	7
6.2	PV 动作	8
6.3	EV 动作	8
7	电池和 EV 的物理状态更新	9
7.1	静态电池 SoC 更新	9
7.2	EV SoC 更新	9

目录	2
8 净负荷、潮流和电网安全	10
8.1 单个用户的净负荷	10
8.2 写入 pandapower 网络	10
8.3 电压约束	10
8.4 线路和变压器约束	11
9 经济项：电池收益、EV 成本和总成本	11
9.1 电池储能收益	11
9.2 EV 充电成本	12
9.3 总成本	12
10 完整算法流程	12
10.1 无电池 Rule-based	12
10.2 有电池 Rule-based	13
11 它和 DRL/MADRL 的区别	13
12 一个数字例子	14
13 阅读代码时应该抓住的主线	15
14 总结	15

1 先建立直觉：这个项目到底在控制什么

本项目研究的是低压配电网中的多个家庭型能源用户。每个用户既会用电，也可能有光伏发电、电池储能和电动汽车充电需求，所以常被称为 prosumer，也就是既是 producer 又是 consumer 的用户。

在每一个离散时间步，控制器要决定三件事：

1. 静态电池是充电、放电，还是不动；
2. 光伏发电是全部使用，还是弃掉一部分；
3. 电动汽车如果在家，应该充多少电。

项目中的强化学习方法会从数据中学习这些决策，而 Rule-based 控制器不学习神经网络。它使用人提前写好的固定规则。因此它通常被当作基线方法：如果一个复杂算法连简单规则都比不过，就说明复杂算法还没有学好。

本文只讲 Rule-based 控制器。它在代码中有两个版本：

- 无电池版本：只控制 EV 充电，不使用静态电池；
- 有电池版本：EV 仍然尽量充电，静态电池根据电价阈值进行低价充电、高价放电。

2 基本符号和单位

为了避免公式看起来像“黑魔法”，先把每个符号的物理意义说明清楚。

符号	单位	含义
i	无	用户编号，项目默认有 $N = 3$ 个用户
t	无	当前时间步
Δt	小时	一个时间步的长度，默认 $\Delta t = 0.25$ ，即 15 分钟
T	步	一天的时间步数，默认 $T = 96$
p_t	欧元/kWh	原始电价
$\tilde{p}_t$	欧元/kWh	实际进口电价，等于原始电价加上加价项
$L_{i,t}$	kW	用户 i 在 t 时刻的本地负荷
$G_{i,t}$	kW	用户 i 在 t 时刻可用的光伏功率
$G_{i,t}^{use}$	kW	实际使用的光伏功率
$P_{i,t}^b$	kW	静态电池功率，正数表示充电，负数表示放电

符号	单位	含义
$P_{i,t}^{ev}$	kW	EV 实际充电功率，只允许非负
$SOC_{i,t}^b$	无	静态电池荷电状态，取值通常在 0 到 1 之间
$SOC_{i,t}^{ev}$	无	EV 电池荷电状态
C_i^b	kWh	静态电池容量，默认每个用户 100 kWh
C_i^{ev}	kWh	EV 电池容量，默认每个用户 60 kWh
$P_i^{b,max}$	kW	静态电池最大充放电功率
$P_i^{ev,max}$	kW	EV 家用充电桩最大功率，默认 11 kW
η_b	无	静态电池效率，默认 0.95
η_{ev}	无	EV 充电效率，默认 0.95

项目中静态电池的最大功率由容量和最大倍率得到：

$$P_i^{b,max} = C_i^b r_{\max}^b. \quad (1)$$

默认 $C_i^b = 100$ kWh, $r_{\max}^b = 0.5$, 所以：

$$P_i^{b,max} = 100 \times 0.5 = 50 \text{ kW}. \quad (2)$$

物理意义是：每个用户的静态电池最多以 50 kW 充电或放电。

3 EV 是否在家的时间规则

EV 不是全天都能充电。项目默认 EV 在 18:00 到家，在次日 07:00 离家。因为一天有 96 个 15 分钟时间步，所以：

$$t_{arr} = 72, \quad t_{dep} = 28. \quad (3)$$

令一天内的步数为：

$$\tau(t) = t \bmod T. \quad (4)$$

由于到家时间 72 晚于离家时间 28，连接窗口跨过午夜。EV 可用指示量可写成：

$$A_t^{ev} = \begin{cases} 1, & \tau(t) \geq t_{arr} \text{ 或 } \tau(t) < t_{dep}, \\ 0, & \text{其他时间}. \end{cases} \quad (5)$$

物理意义是：晚上 18:00 到午夜、午夜到早上 07:00，EV 在家，可以充电；白天离家，不能充电。

EV 到家时，项目把 EV SoC 重置为到家 SoC：

$$SOC_{i,t}^{ev} = SOC_{arr}^{ev}. \quad (6)$$

默认:

$$SOC_{arr}^{ev} = 0.30. \quad (7)$$

这表示 EV 回家时还有 30% 的电。离家目标是:

$$SOC_{req}^{ev} = 0.90. \quad (8)$$

也就是希望第二天早上离家时至少有 90% 的电。

4 Rule-based 无电池版本

4.1 核心思想

无电池版本只做一件事: 只要 EV 在家, 并且 EV 没满, 就用最大功率充电。静态电池不参与调度。

静态电池功率恒为:

$$P_{i,t}^b = 0. \quad (9)$$

光伏弃光也恒为:

$$G_{i,t}^{curt} = 0. \quad (10)$$

因此实际使用的光伏功率是:

$$G_{i,t}^{use} = G_{i,t}. \quad (11)$$

EV 充电规则为:

$$P_{i,t}^{ev} = \begin{cases} P_i^{ev, \max}, & A_t^{ev} = 1 \text{ 且 } SOC_{i,t}^{ev} < SOC_{\max}^{ev}, \\ 0, & \text{其他情况.} \end{cases} \quad (12)$$

这里 $SOC_{\max}^{ev}$ 默认是 0.95。物理意义是: EV 在家且没充到上限时, 就满功率充; 如果 EV 不在家, 或者已经接近满电, 就不充。

4.2 为什么这个规则简单但不聪明

这个规则不看电价, 也不看未来。比如凌晨 2 点电价很低, 晚上 19 点电价很高, 它也可能刚到家就高价充电, 因为规则只问“在不在家、满没满”, 不问“现在贵不贵”。

因此无电池 Rule-based 通常能满足充电需求, 但经济性不一定好, 也不会主动缓解电网压力。

5 Rule-based 有电池版本

5.1 核心思想

有电池版本在无电池版本的基础上加入静态电池规则:

- 电价低时，电池充电；
- 电价高时，电池放电；
- 电价处于中间区域时，电池不动。

这就是最朴素的套利思想。低价买电存进电池，高价时从电池放出，减少高价购电，甚至形成储能收益。

5.2 进口电价

控制器使用的价格不是只看原始电价 p_t ，而是：

$$\tilde{p}_t = p_t + \mu_{import}. \quad (13)$$

其中 μ_{import} 是进口电价加价项。默认情况下它为 0，所以 $\tilde{p}_t = p_t$ 。

5.3 价格分位数阈值

控制器先从训练集电价中计算两个阈值。设训练集进口电价序列为：

$$\mathcal{P}_{train} = \{\tilde{p}_1, \tilde{p}_2, \dots, \tilde{p}_M\}. \quad (14)$$

低价阈值为：

$$p^{low} = Q_{q_l}(\mathcal{P}_{train}), \quad (15)$$

高价阈值为：

$$p^{high} = Q_{q_h}(\mathcal{P}_{train}), \quad (16)$$

其中 $Q_q(\cdot)$ 表示 q 分位数。项目中的 MPC 配置默认使用：

$$q_l = 0.35, \quad q_h = 0.65. \quad (17)$$

代码也支持 Rule-based 专用分位数参数；若未设置专用参数，就使用 MPC 中的低价和高价分位数。

分位数的物理意义很直观：

- p^{low} 以下代表相对便宜的电；
- p^{high} 以上代表相对昂贵的电；
- 中间区域代表价格不够极端，暂时不操作电池。

5.4 电池功率规则

令 α_c 为充电比例, α_d 为放电比例, 二者都被限制在 $[0, 1]$ 。默认都是 1, 即可以用最大功率操作。Rule-based 有电池版本的功率请求是:

$$P_{i,t}^{b,req} = \begin{cases} \alpha_c P_i^{b,max}, & \tilde{p}_t \leq p^{low}, \\ -\alpha_d P_i^{b,max}, & \tilde{p}_t \geq p^{high}, \\ 0, & p^{low} < \tilde{p}_t < p^{high}. \end{cases} \quad (18)$$

注意正负号:

- $P^b > 0$ 表示电池充电, 电网侧负荷增加;
- $P^b < 0$ 表示电池放电, 电网侧负荷减少。

如果默认 $\alpha_c = \alpha_d = 1$, 且每个电池最大功率为 50 kW, 那么:

$$P_{i,t}^{b,req} = \begin{cases} 50, & \text{低价时充电,} \\ -50, & \text{高价时放电,} \\ 0, & \text{中间价不动.} \end{cases} \quad (19)$$

5.5 备用规则: 中位数两段式

代码中还有一个可选模式: 如果关闭中间空闲区间, 就用训练集价格中位数 p^{med} 分成两类:

$$P_{i,t}^{b,req} = \begin{cases} \alpha_c P_i^{b,max}, & \tilde{p}_t \leq p^{med}, \\ -\alpha_d P_i^{b,max}, & \tilde{p}_t > p^{med}. \end{cases} \quad (20)$$

这个模式会让电池几乎一直动作。它更激进, 但也更容易造成无意义循环和电网压力。当前主线规则默认保留中间空闲区间。

6 从功率请求到环境动作

环境真正接收的是动作数组, 而不是直接接收 kW 功率。Rule-based 控制器先算出功率请求, 然后调用动作映射函数。

6.1 电池动作

电池原始动作可以理解为归一化功率:

$$a_{i,t}^b = \frac{P_{i,t}^{b,req}}{P_i^{b,max}}. \quad (21)$$

理论上 $a_{i,t}^b \in [-1, 1]$ 。但是环境还会根据当前 SoC 做可行性裁剪，防止一步之后越过电池 SoC 上下界。

静态电池 SoC 下界和上界为：

$$SOC_{\min}^b = 0.05, \quad SOC_{\max}^b = 0.95. \quad (22)$$

给定当前 $SOC_{i,t}^b$ ，动作上界为：

$$\bar{a}_{i,t}^b = \frac{(SOC_{\max}^b - SOC_{i,t}^b)C_i^b}{P_i^{b,\max} \Delta t \eta_b}. \quad (23)$$

它的物理意义是：当前电池还能装多少能量，折算成这个时间步内最多允许多大的充电动作。

动作下界为：

$$\underline{a}_{i,t}^b = \frac{(SOC_{\min}^b - SOC_{i,t}^b)\eta_b C_i^b}{P_i^{b,\max} \Delta t}. \quad (24)$$

它通常是负数。物理意义是：当前电池还能放出多少能量，折算成这个时间步内最多允许多大的放电动作。

最终执行动作是：

$$a_{i,t}^{b,exec} = \text{clip} \left(a_{i,t}^b, \max(\underline{a}_{i,t}^b, -1), \min(\bar{a}_{i,t}^b, 1) \right). \quad (25)$$

执行功率为：

$$P_{i,t}^b = a_{i,t}^{b,exec} P_i^{b,\max}. \quad (26)$$

6.2 PV 动作

Rule-based 控制器不主动弃光，所以：

$$G_{i,t}^{curt} = 0, \quad G_{i,t}^{use} = G_{i,t}. \quad (27)$$

如果写成 PV 利用率：

$$u_{i,t}^{pv} = \frac{G_{i,t}^{use}}{G_{i,t}}. \quad (28)$$

在 $G_{i,t} > 0$ 时，Rule-based 的 $u_{i,t}^{pv} = 1$ 。环境动作里使用 $[-1, 1]$ 表示，所以：

$$a_{i,t}^{pv} = 2u_{i,t}^{pv} - 1 = 1. \quad (29)$$

6.3 EV 动作

EV 充电功率先转成利用率：

$$u_{i,t}^{ev} = \text{clip} \left(\frac{P_{i,t}^{ev}}{P_i^{ev,\max}}, 0, 1 \right). \quad (30)$$

再映射到环境动作：

$$a_{i,t}^{ev} = 2u_{i,t}^{ev} - 1. \quad (31)$$

如果 EV 满功率充电， $u^{ev} = 1$ ，所以 $a^{ev} = 1$ ；如果不充电， $u^{ev} = 0$ ，所以 $a^{ev} = -1$ 。

7 电池和 EV 的物理状态更新

7.1 静态电池 SoC 更新

环境中电池功率正数表示充电，负数表示放电。由于效率存在，充电和放电的 SoC 更新不同。

当 $P_{i,t}^b \geq 0$ 时，电池充电：

$$SOC_{i,t+1}^b = SOC_{i,t}^b + \frac{P_{i,t}^b \Delta t \eta_b}{C_i^b}. \quad (32)$$

物理意义是：从电网吸收的能量为 $P^b \Delta t$ ，只有 η_b 的比例真正存进电池。

当 $P_{i,t}^b < 0$ 时，电池放电：

$$SOC_{i,t+1}^b = SOC_{i,t}^b + \frac{P_{i,t}^b \Delta t}{\eta_b C_i^b}. \quad (33)$$

因为 $P_{i,t}^b$ 是负数，所以 SoC 会下降。除以 η_b 的物理意义是：为了向外提供一定电能，电池内部要消耗更多能量。

最后环境裁剪到安全范围：

$$SOC_{i,t+1}^b \leftarrow \text{clip}(SOC_{i,t+1}^b, SOC_{\min}^b, SOC_{\max}^b). \quad (34)$$

7.2 EV SoC 更新

EV 只有充电，没有向电网放电。EV 实际充电功率还会受到剩余容量限制：

$$E_{i,t}^{\text{head}} = \max(0, SOC_{\max}^{\text{ev}} - SOC_{i,t}^{\text{ev}}) C_i^{\text{ev}}. \quad (35)$$

这是 EV 距离上限还能存下的能量，单位 kWh。

本时间步最多允许的充电功率为：

$$P_{i,t}^{\text{ev,head}} = \frac{E_{i,t}^{\text{head}}}{\Delta t \eta_{\text{ev}}}. \quad (36)$$

最终执行的 EV 充电功率为：

$$P_{i,t}^{\text{ev,exec}} = \min(P_{i,t}^{\text{ev,req}}, P_{i,t}^{\text{ev,head}}). \quad (37)$$

EV SoC 更新为：

$$SOC_{i,t+1}^{\text{ev}} = \text{clip} \left(SOC_{i,t}^{\text{ev}} + \frac{P_{i,t}^{\text{ev,exec}} \Delta t \eta_{\text{ev}}}{C_i^{\text{ev}}}, SOC_{\min}^{\text{ev}}, SOC_{\max}^{\text{ev}} \right). \quad (38)$$

物理意义是：EV 从充电桩得到功率，乘以时间得到电能，再乘以效率后才真正进入 EV 电池。

8 净负荷、潮流和电网安全

8.1 单个用户的净负荷

环境把所有设备作用合成用户的净负荷：

$$P_{i,t}^{net} = L_{i,t} - G_{i,t}^{use} + P_{i,t}^b + P_{i,t}^{ev}. \quad (39)$$

每一项的含义是：

- $L_{i,t}$ 是家庭本来要用的电，会增加净负荷；
- $G_{i,t}^{use}$ 是本地光伏，会抵消负荷；
- $P_{i,t}^b > 0$ 时电池充电，会增加净负荷；
- $P_{i,t}^b < 0$ 时电池放电，会减少净负荷；
- $P_{i,t}^{ev}$ 是 EV 充电，会增加净负荷。

如果 $P_{i,t}^{net} > 0$ ，表示该用户总体从电网取电。如果 $P_{i,t}^{net} < 0$ ，表示该用户向电网注入功率。

8.2 写入 pandapower 网络

环境会把 $P_{i,t}^{net}$ 写入电网模型。若净负荷为正，就写入负荷表：

$$P_{i,t}^{load} = \max(0, P_{i,t}^{net}). \quad (40)$$

若净负荷为负，就写入分布式发电表：

$$P_{i,t}^{sgen} = \max(0, -P_{i,t}^{net}). \quad (41)$$

注意代码单位要从 kW 转成 MW：

$$P_{MW} = \frac{P_{kW}}{1000}. \quad (42)$$

然后 pandapower 求解交流潮流，得到母线电压、线路负载率和变压器负载率。

8.3 电压约束

项目默认电压允许范围是：

$$v_{\min} = 0.95, \quad v_{\max} = 1.05. \quad (43)$$

单位是 p.u.，即标么值。1.0 p.u. 表示额定电压，0.95 表示额定值的 95%，1.05 表示额定值的 105%。

某个母线的电压越界量为：

$$\xi_{k,t}^v = \max(0, v_{\min} - v_{k,t}) + \max(0, v_{k,t} - v_{\max}). \quad (44)$$

如果 $v_{k,t}$ 在安全范围内，两个最大值都是 0；如果低电压或过电压，就产生正的越界量。电压安全惩罚原始量为：

$$\psi_t^v = \sum_k (\xi_{k,t}^v)^2. \quad (45)$$

8.4 线路和变压器约束

线路负载率和变压器负载率用百分比表示。默认安全上限是 100%。线路越界量为：

$$\xi_{m,t}^\ell = \frac{\max(0, \ell_{m,t} - \ell_{\max})}{100}. \quad (46)$$

线路安全惩罚原始量为：

$$\psi_t^\ell = \sum_m (\xi_{m,t}^\ell)^2. \quad (47)$$

变压器越界量为：

$$\xi_{r,t}^{tr} = \frac{\max(0, tr_{r,t} - tr_{\max})}{100}. \quad (48)$$

变压器安全惩罚原始量为：

$$\psi_t^{tr} = \sum_r (\xi_{r,t}^{tr})^2. \quad (49)$$

物理意义是：越界越严重，平方惩罚增长越快。因此轻微越界和严重越界在评价中会被明显区分。

9 经济项：电池收益、EV 成本和总成本

9.1 电池储能收益

环境中的电池收益为：

$$R_{i,t}^b = [\max(-P_{i,t}^b, 0) - \max(P_{i,t}^b, 0)] \tilde{p}_t \Delta t. \quad (50)$$

分两种情况理解：

- 充电时 $P^b > 0$ ，收益为 $-P^b \tilde{p}_t \Delta t$ ，是负数，表示买电成本；
- 放电时 $P^b < 0$ ，收益为 $(-P^b) \tilde{p}_t \Delta t$ ，是正数，表示避免高价购电或等价收入。

所以 Rule-based 有电池版本的目标直觉是：低价时接受负收益存电，高价时获得正收益放电。只要高价收益大于低价成本，就有套利价值。

9.2 EV 充电成本

EV 充电成本为:

$$C_{i,t}^{ev} = w_{ev} P_{i,t}^{ev} \tilde{p}_t \Delta t. \quad (51)$$

其中 w_{ev} 是 EV 成本权重。对 Rule-based 基线而言，可以把它理解为实际电费的缩放系数。若 $w_{ev} = 1$ ，就是普通电费。

EV 只充电不放电，所以它没有“收益”项。它的充电越多、价格越高，成本越大。

9.3 总成本

结果统计中常用的总成本可以写成:

$$C^{total} = \sum_{t,i} (-R_{i,t}^b + C_{i,t}^{ev} + C_{i,t}^{pv}). \quad (52)$$

当前 PV 成本通常为 0，除非额外配置弃光价格或惩罚。因此可以近似理解为:

$$C^{total} \approx \sum_{t,i} (-R_{i,t}^b + C_{i,t}^{ev}). \quad (53)$$

这里 $-R^b$ 是电池成本。如果电池放电赚钱， $R^b > 0$ ，那么 $-R^b < 0$ ，会降低总成本。

10 完整算法流程

10.1 无电池 Rule-based

Listing 1: 无电池 Rule-based 控制流程

```

对每个时间步 t:
  对每个用户 i:
    P_b[i] = 0
    PV_curtail[i] = 0

    如果 EV 在家 且 EV_SOC[i] < EV_SOC_max:
      P_ev[i] = EV_Pmax[i]
    否则:
      P_ev[i] = 0

  把 P_b、PV_curtail、P_ev 转成环境动作
  环境更新 SoC、净负荷、潮流和成本

```

10.2 有电池 Rule-based

Listing 2: 有电池 Rule-based 控制流程

先用训练集电价计算：

```
low_threshold = 低价分位数
high_threshold = 高价分位数
```

对每个时间步 t ：

```
current_price = 当前电价 + 进口加价
```

对每个用户 i ：

```
如果 current_price <= low_threshold:
    P_b[i] = +charge_fraction * P_b_max[i]
否则如果 current_price >= high_threshold:
    P_b[i] = -discharge_fraction * P_b_max[i]
否则:
    P_b[i] = 0
```

```
PV_curtail[i] = 0
```

```
如果 EV 在家 且 EV_SOC[i] < EV_SOC_max:
```

```
    P_ev[i] = EV_Pmax[i]
```

```
否则:
```

```
    P_ev[i] = 0
```

环境按 SoC 上下界裁剪电池动作

环境按 EV 剩余容量裁剪 EV 充电

环境更新 SoC、净负荷、潮流和成本

11 它和 DRL/MADRL 的区别

Rule-based 和 DRL/MADRL 最大的区别不是“公式多不多”，而是决策来源不同。

Rule-based 的决策来源是人写好的规则：

$$\text{状态} \rightarrow \text{固定判断条件} \rightarrow \text{动作}. \quad (54)$$

MADRL 的决策来源是训练出来的策略网络：

$$\text{状态} \rightarrow \pi_{\theta}(\text{状态}) \rightarrow \text{动作}. \quad (55)$$

Rule-based 不需要训练，计算很快，也很容易解释。但它有几个明显局限：

- 它只看当前电价，不真正优化未来多个时间步；

- EV 规则不看电价，所以可能高价充电；
- 所有用户看到同一个电价阈值，容易同时充电或同时放电，造成电网压力；
- 它不主动根据电压、线路负载率、变压器负载率调整动作；
- 它不会从历史失败中学习，也不会适应复杂的负荷、PV 和 EV 模式。

因此，Rule-based 的意义主要是提供一个清楚、简单、可解释的比较对象。复杂算法如果表现更好，应当能解释为：它更好地利用了未来预测、设备耦合、电价变化和电网安全反馈。

12 一个数字例子

假设某个用户的参数为：

$$C^b = 100 \text{ kWh}, \quad P^{b,\max} = 50 \text{ kW}, \quad \Delta t = 0.25 \text{ h}, \quad \eta_b = 0.95. \quad (56)$$

如果低价时满功率充电：

$$P^b = 50 \text{ kW}. \quad (57)$$

本步存入电池的能量为：

$$50 \times 0.25 \times 0.95 = 11.875 \text{ kWh}. \quad (58)$$

SoC 增加：

$$\Delta SOC^b = \frac{11.875}{100} = 0.11875. \quad (59)$$

也就是电池 SoC 增加约 11.875 个百分点。

如果高价时满功率放电：

$$P^b = -50 \text{ kW}. \quad (60)$$

SoC 变化为：

$$\Delta SOC^b = \frac{-50 \times 0.25}{0.95 \times 100} \approx -0.1316. \quad (61)$$

也就是电池 SoC 下降约 13.16 个百分点。放电时 SoC 降得更多，是因为效率损耗使电池内部要消耗更多能量。

再看 EV。若 EV 以 11 kW 充 15 分钟，容量 60 kWh，效率 0.95：

$$\Delta SOC^{ev} = \frac{11 \times 0.25 \times 0.95}{60} \approx 0.0435. \quad (62)$$

也就是 EV SoC 增加约 4.35 个百分点。

13 阅读代码时应该抓住的主线

如果你刚入门，不建议一开始就试图读懂所有文件。可以按下面的顺序理解：

1. 先读控制器：Rule-based 怎样把当前状态变成功率请求；
2. 再读动作映射：功率请求如何变成环境动作；
3. 再读环境 step：动作如何更新 SoC、EV、PV 和净负荷；
4. 再读潮流核心：净负荷如何写进 pandapower 并得到电压、线路、变压器结果；
5. 最后读记录和评估：一次 rollout 如何被转成成本表、图和指标。

对应到当前项目，关键文件是：

- `controllers/rule_based.py`: Rule-based 控制规则；
- `envs/grid_env.py`: 动作执行、SoC 更新、成本和奖励计算；
- `envs/grid_core.py`: pandapower 潮流计算和电网安全指标；
- `scripts/rule_based.py`: 运行 Rule-based 基线、保存表格和图；
- `configs/cfg.py`: 默认参数。

14 总结

Rule-based 控制器可以概括为一句话：

EV 在家且没满就满功率充；如果启用静态电池，则电价低于低分位阈值时充电，高于高分位阈值时放电，中间价格不动作。

它的数学结构很简单，但背后仍然连接了完整的物理系统：EV SoC、静态电池 SoC、光伏利用、净负荷、交流潮流、电压越界、线路越载、变压器越载和经济成本。

对于初学者来说，理解 Rule-based 的价值在于：它先给出一个不需要学习的清晰基准。之后再理解 MADRL 时，就可以问一个更具体的问题：学习算法究竟学到了什么，是不是比“低价充、高价放、EV 满功率充”这个简单策略更会利用未来信息和系统耦合。